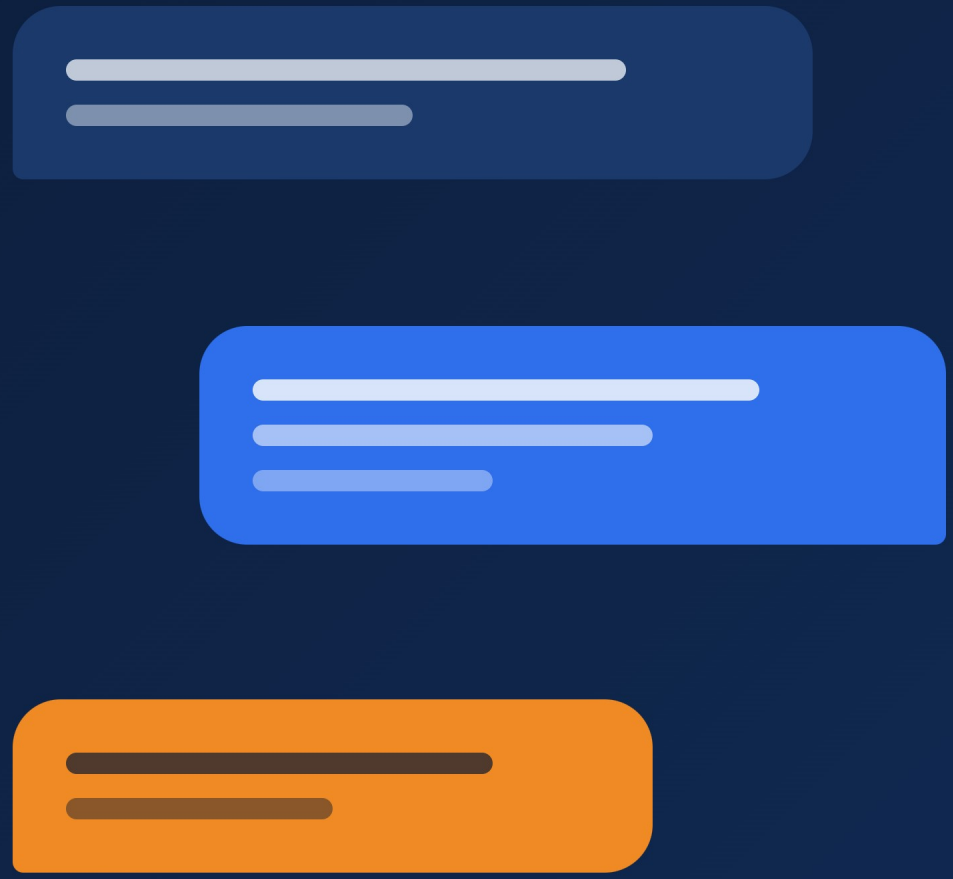
RESTFUL API · NODE.JS · EXPRESS · MONGODB ATLAS

Sarahah_App

A RESTful API for anonymous messaging and user management, with authentication, account management and secure message handling.

Mustafa M. Abdelaziz

GitHub: MustafaDols



Contents

Four parts, from the big picture down to the details.

01 **System Overview**
Purpose, stack and architecture

02 **Authentication**
Tokens, roles and the sign-in flow

03 **API Endpoints**
Users, accounts and messaging

04 **Security & Middleware**
Protection layers and the pipeline

01 · SYSTEM OVERVIEW

Purpose and Technology Stack

Sarhah_App is a RESTful API for anonymous messaging and user management. It covers authentication, account management and secure message handling.

Node.js
Runtime for the backend

Express.js
REST framework with modular routes

MongoDB Atlas
Stores users and messages

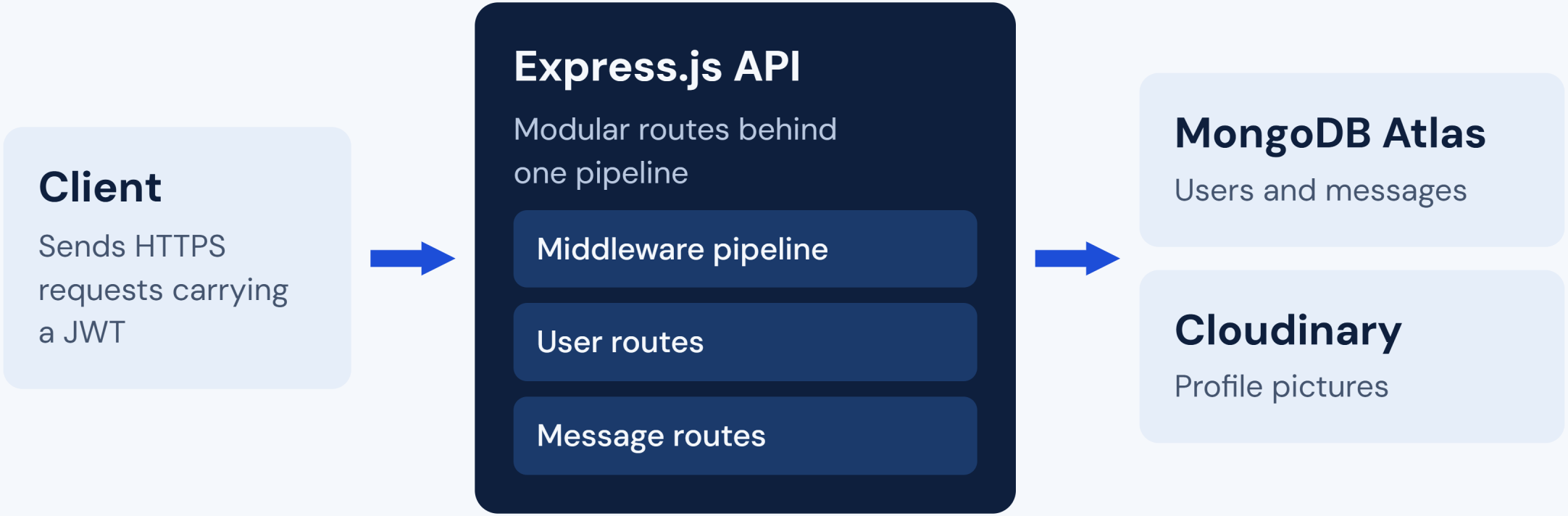
JWT
Stateless access and refresh tokens

Joi
Schema validation for requests

Cloudinary
Profile picture storage with CDN

01 · SYSTEM OVERVIEW

System Architecture



Authentication System Design

JWT Token System

Stateless access and refresh tokens for sessions and API access

Role-Based Access

Super Admin, Admin and User roles control access to resources

Multi-Provider Auth

Google OAuth alongside email and password sign-in

Security Middleware

Rate limiting, input validation and CORS against common attacks

Sign-In Flow

1	**Sign up**	`POST /users/signup`	Register with email validation
2	**Confirm email**	`PUT /users/confirm`	Verify with a confirmation token
3	**Sign in**	`POST /users/signin`	Receive access and refresh JWTs
4	**Refresh**	`POST /users/refresh-token`	New access token every 15 minutes
5	**Log out**	`POST /users/logout`	Invalidate session and tokens

User Endpoints

Endpoint	Method	Auth required	Description
/users/signup	POST	No	Register a new account with email validation
/users/signin	POST	No	Authenticate and return JWT tokens
/users/confirm	PUT	No	Verify the email via confirmation token
/users/logout	POST	Yes	Invalidate the session and tokens
/users/refresh-token	POST	No	Issue a new access token from a refresh token

Access tokens expire after 15 minutes; the refresh token issues a new one.

Account Management Endpoints

Endpoint	Method	Auth required	Description
/users/update	PUT	Yes	Update profile information
/users/updatePassword	PUT	Yes	Change the account password
/users/resetPassword	PUT	No	Reset the password via an email token
/users/upload-profile	POST	Yes	Upload a profile picture to Cloudinary
/users/delete/:userId	DELETE	Yes	Delete the account permanently

Profile pictures are limited to 5 MB and stored in Cloudinary.

Messaging Endpoints

Endpoint	Method	Auth required	Description
/messages/send/:receiverId	POST	Yes	Send a direct message to a user
/messages/get/:receiverId	GET	Yes	Retrieve conversation history with a user
/messages/	GET	Yes	List all conversations
/messages/unread	GET	Yes	Count unread messages in all conversations

Across users, accounts and messaging the API exposes 14 endpoints; every messaging endpoint requires authentication.

04 · SECURITY & MIDDLEWARE

Security Implementation

100%
HTTPS only

15 min
Token expiry

5 MB
File upload limit

10 req/min
Rate limit

Layered protection across authentication, the API and file handling.

Three Layers of Protection

Authentication

- bcrypt password hashing with salt rounds
- JWT expiration and refresh mechanism
- Secure HTTP-only cookie storage

API Protection

- Rate limiting per IP and user
- Input validation with Joi schemas
- CORS whitelist configuration

File Security

- File type validation and size limits
- Cloudinary secure storage with CDN
- Automatic old file deletion

Middleware Pipeline Flow

1	**Security Layer**	Helmet.js headers, CORS and rate limiting
2	**Authentication**	JWT validation and session management with token refresh
3	**Validation**	Joi schemas validate the request body
4	**File Processing**	Multer uploads, Cloudinary and cleanup services

Thank You

Questions & discussion welcome

Mustafa M. Abdelaziz · GitHub: MustafaDols